

OWL-AGENT v4.2 + LLM-DNS-Proxy

Synergy Merger Analysis & Implementation Blueprint

A/B Comparison Matrix | Unified Architecture | Obsidian Integration

Operating Protocol: SMP-v5.1 (Silent Methodological Protocol)

Deep Research & Orchestration | Skills.sh Discovery | End-to-End Simulation

Generated: 2026-07-28 | Z.ai Research Division

Table of Contents

1. Executive Summary & SMP-v5.1 Adoption
2. Skills Discovery & Capability Composition
3. Project A: OWL-AGENT v4.2 Architecture Deep Dive
4. Project B: LLM-DNS-Proxy Architecture Deep Dive
5. A/B Comparison Matrix (Pros, Cons, Layers, Tools, Components, Star Ratings)
6. Unified Architecture & Synergy Analysis
7. Implementation Plan (Phased Roadmap)
8. Unified Synergies Script
9. Obsidian Integration via kepano/obsidian-skills
10. Appendix: Skills.sh Trending & Scraper Ecosystem

1. Executive Summary & SMP-v5.1 Adoption

This document presents a comprehensive merger analysis of two powerful network-utility projects: **OWL-AGENT v4.2** (a self-optimizing scraping engine with 50+ proxy sources, quality scoring, adaptive rate limiting, and Chrome fingerprinting) and **LLM-DNS-Proxy** (a DNS tunneling system that enables encrypted LLM conversations through captive portals and restricted networks). The analysis follows the **SMP-v5.1 Silent Methodological Protocol** as the governing operating instructions, which mandates zero-fluff execution, alignment over speed, quality-gated reasoning, and depth before velocity.

The SMP-v5.1 protocol establishes a state-machine workflow with hard gates at every stage: Discovery, Brainstorming, Research, Planning, Execution, Validation, Review, and Completion. Each stage must be satisfied before proceeding. The protocol also routes cognitive effort through seven distinct modes: Rabbit (speed), Ant (systematic), Beaver (builder), Owl (depth), Eagle (strategy), Dolphin (creative), and Elephant (memory). For this merger analysis, we primarily employ Owl (depth analysis of hidden factors) and Beaver (practical system design), with Ant for incremental task decomposition and Eagle for strategic pattern recognition across both projects.

The core insight driving this merger is that OWL-AGENT and LLM-DNS-Proxy occupy complementary positions in the network-access stack. OWL-AGENT excels at **getting through** (bypassing anti-bot systems, rotating proxies, fingerprinting browsers) while LLM-DNS-Proxy excels at **getting under** (tunneling through DNS when HTTP is blocked entirely). Combined, they create a dual-channel resilient system: when HTTP proxies fail, DNS tunneling activates; when DNS is monitored, HTTP proxy rotation takes over. This creates a self-healing, multi-protocol access layer that no single project can achieve alone.

1.1 Key Findings

- **Complementary Access Channels:** OWL-AGENT provides HTTP/HTTPS proxy evasion; LLM-DNS-Proxy provides DNS tunnel evasion. Together they cover the two primary network bypass vectors.
- **Shared Intelligence Layer:** Both projects implement quality scoring, adaptive rate limiting, and retry logic. A unified scorer can combine DNS-latency metrics with HTTP-success-rate metrics.
- **Session Persistence:** OWL-AGENT uses Redis for state sharing; LLM-DNS-Proxy uses session persistence for conversation context. A merged RedisStore can serve both purposes.
- **Obsidian Integration:** Both projects can output scraped/tunneled data as Markdown, making them natural fits for Obsidian vaults via kepano/obsidian-skills.
- **Star Rating Preview:** OWL-AGENT scores 4.5/5 on proxy management but 2/5 on restricted-network bypass; LLM-DNS-Proxy scores 5/5 on DNS evasion but 2/5 on HTTP resilience. Merged system targets 4.5/5 across all dimensions.

2. Skills Discovery & Capability Composition

Following SMP-v5.1 Stage 1 (Discovery & Capability Composition), we browsed `skills.sh/trending` to identify available agent skills, then composed the bespoke toolset needed for this specific task. The `skills.sh` marketplace hosts skills for Claude Code, Cursor, Windsurf, OpenCode, and other AI agents, with trending categories including browser automation, web scraping, data processing, and Obsidian integration. The `find-skills` command on `skills.sh` enables version-locked discovery and installation of relevant capabilities.

2.1 Skills.sh Trending Landscape

The `skills.sh` trending page reveals the fastest-growing AI agent skills each week, categorized by agent compatibility. The marketplace includes skills for browser automation (`agent-browser` by `vercel-labs`), Obsidian integration (`obsidian-skills` by `kepano`), web research (`parallel-deep-research`), code generation, and DevOps automation. For this merger task, we identified the following version-locked toolset:

Skill / Tool	Version	Role in Merger	Source
<code>find-skills (skills.sh)</code>	<code>latest</code>	Discover and install scraper/research skills	<code>npx skills find scraper</code>
<code>agent-browser (vercel-labs)</code>	<code>v2.x</code>	Headless browser for OWL-AGENT JS rendering	<code>npx skills add agent-browser</code>
<code>obsidian-skills (kepano)</code>	<code>v1.x</code>	Obsidian CLI + Markdown/Canvas/Bases integration	<code>npx skills add kepano/obsidian-skills</code>
<code>parallel-deep-research</code>	<code>v1.x</code>	Multi-source deep research for proxy pool analysis	<code>npx skills add parallel-deep-research</code>
<code>web-reader (z-ai)</code>	<code>v1.x</code>	Content extraction from web pages	<code>z-ai-web-dev-sdk</code>
<code>page_reader</code>	<code>v1.x</code>	Raw page content extraction (GitHub READMEs)	<code>github-readme-extractor</code>

Table 2.1: Version-locked toolset assembled for this merger task

2.2 Scraper Skills Discovery

Running `npx skills find scraper` on `skills.sh` yields a catalog of scraping-related skills. The most relevant ones for this merger include skills for proxy pool management, DNS-based data retrieval, browser automation with fingerprint evasion, and Markdown conversion for Obsidian ingestion. The scraper ecosystem on `skills.sh` spans three categories: (1) HTTP-based scrapers that use proxy rotation and TLS fingerprinting (where OWL-AGENT belongs), (2) protocol-level scrapers that tunnel through DNS or ICMP (where LLM-DNS-Proxy belongs), and (3) hybrid scrapers that combine both approaches (the target of our merged system).

The key discovery from `skills.sh` is that no existing skill combines HTTP proxy evasion with DNS tunneling in a single unified interface. This gap is precisely what the OWL-AGENT + LLM-DNS-Proxy merger fills. The merged skill would be installable via `npx skills add owl-dns-synergy` and would expose both access channels through a single `ResilientClient` API, automatically selecting the optimal channel based on network conditions and target accessibility.

3. Project A: OWL-AGENT v4.2

Architecture Deep Dive

OWL-AGENT v4.2 is a production-grade, self-healing HTTP client that combines over 50 proxy sources, quality scoring, adaptive rate limiting, Redis state sharing, curl_cffi Chrome fingerprinting, Retry-After parsing, circuit breaker protection, and agent-browser automation into a single memory-efficient Python script. It is designed for high-volume SEO scraping, API integration, and dynamic content retrieval, with seamless integration into modern AI-coding ecosystems including OpenCode, Cline, Cursor, Warp, Codebuff, Claude Code, Codex, Antigravity, Kiro-CLI, Hermes-Agent, and Obsidian Skills.

3.1 Layer Architecture

Layer	Components	Function	Why It Matters
User Layer	OpenCode, Cline, Cursor, Warp, Claude Code, Google AI Interface, Kiro-CLI, Hermes-Agent	Human/agent interaction, invoke OWL-AGENT	Users stay in preferred environment
API Interface	CLI (run.sh), Python class, OpenCode Exposer	OWL-AGENT as a reusable tool	Both human and agentic usage
Core Engine	ResilientClient orchestrator	Caching, dedup, rate limiting, scoring, pacing, circuit breaker	Central logic & proxy selection
Proxy Management	ProxyPoolManager + ProxyBroker2	50+ proxy sources, country filtering, validation	Self-healing proxy supply
Intelligence	QualityScorer, AdaptiveRateLimiter, ProxyBreaker	Proxy scoring, request pacing, failure thresholds	Adapt & improve success
Persistence	RedisStore (optional)	Shares proxy pool, scores, circuit state across hosts	Horizontal scaling
Fingerprint Evasion	curl_cffi (Chrome 110 impersonation)	Mimics real browser TLS handshake	Bypasses anti-bot detection
Headless Browser	agent-browser	JavaScript-heavy page rendering	Handles SPAs, captchas
Obsidian Integration	obsidian-skills (kepano)	Read/write Obsidian notes, bases, markdown	Knowledge base as data source

Table 3.1: OWL-AGENT v4.2 layer architecture

3.2 Core Components Deep Dive

3.2.1 HTTPCache (LRU + Disk)

The HTTPCache implements a two-tier caching system: an in-memory LRU dictionary for fast lookups and a disk-based JSON store at `~/.owl-agent/cache/http/` for persistence across restarts. Each cached response includes status code, content bytes, headers, timestamp, TTL, and protocol. A background cleanup loop runs every 60 seconds, evicting expired entries and trimming oversized caches to the configured maximum (default 1000 entries). Cache keys are SHA-256 hashes of method + URL + parameters + protocol, ensuring deterministic deduplication even across different HTTP/2 vs HTTP/1.1 sessions.

3.2.2 RequestDeduplicator

The RequestDeduplicator prevents redundant concurrent requests by tracking in-flight futures keyed by the same SHA-256 hash used by the cache. When two requests arrive simultaneously for the same resource, the second request awaits the first request's future rather than issuing a duplicate network call. This is critical for high-concurrency scraping scenarios where multiple threads or agents may target the same URL simultaneously. The deduplicator uses `asyncio.Lock` for thread-safe registration and cleanup of futures, with proper exception propagation to all waiting consumers.

3.2.3 QualityScorer

The QualityScorer implements an exponential moving average (EMA) with configurable decay factor (default 0.9) to track proxy quality over time. Each proxy receives a score between 0.0 (completely unreliable) and 1.0 (perfect reliability), updated on every request outcome. The EMA formula is: $\text{new_score} = \text{old_score} * \text{decay} + (\text{success} ? 1.0 : 0.0) * (1 - \text{decay})$. This ensures recent failures weigh heavily while historical performance provides stability. The scorer also maintains a latency history (last 100 measurements per proxy) for latency-aware proxy selection. The `get_best_proxy()` method selects the proxy with the highest combined score from a candidate pool.

3.2.4 AdaptiveRateLimiter

The AdaptiveRateLimiter dynamically adjusts per-domain request rates based on server response codes. On success (200-299), the rate increases by 10% toward a configurable maximum (default 5.0 req/s). On rate-limit responses (429, 503), the rate decreases by 50% toward a configurable minimum (default 0.1 req/s). Other error codes maintain the current rate. This creates a natural throttling behavior that automatically backs off when servers indicate overload, and gradually ramps up when servers are responsive. Each domain maintains its own independent rate, preventing a single slow domain from bottlenecking others.

3.2.5 CircuitBreaker

The CircuitBreaker follows the standard three-state pattern (Closed, Open, Half-Open) with configurable failure threshold (default 5 consecutive failures) and recovery timeout (default 30 seconds). When a domain exceeds the failure threshold, the circuit opens and all subsequent requests to that domain are immediately rejected without network calls, saving resources and preventing cascading failures. After the recovery timeout, the circuit enters half-open state and allows a single probe request. If the probe succeeds, the circuit closes and normal traffic resumes. This pattern is essential for scraping scenarios where target sites may temporarily go down or deploy aggressive anti-bot measures.

3.2.6 ProxyPoolManager

The ProxyPoolManager continuously discovers proxies from 50+ sources via ProxyBroker2, filters them by country (default: US, GB, DE, FR, CA), validates connectivity, and maintains a rotating pool. Proxies are tracked as ProxyEntry objects with health status, failure counts, and ban timers. Failed proxies are automatically banned for 60 seconds and retried later. The manager supports LitProxy for rotation and integrates with curl_cffi for Chrome 110 TLS impersonation. When RedisStore is enabled, the proxy pool state is shared across multiple OWL-AGENT instances, enabling distributed scraping with zero proxy wastage.

4. Project B: LLM-DNS-Proxy Architecture Deep Dive

LLM-DNS-Proxy is a DNS tunneling system that enables encrypted LLM (Large Language Model) conversations through captive portals, corporate firewalls, and network filters. It exploits a fundamental oversight in network restrictions: while most traffic (HTTP, HTTPS) gets blocked until authentication, DNS queries almost always work because captive portals themselves need DNS to function. The project masks encrypted chat messages as mundane DNS lookups (e.g., mDNS device discovery traffic), chunking and encrypting data across multiple DNS TXT record queries that pass through unrestricted.

4.1 Protocol Architecture

The DNS tunneling protocol operates in seven distinct phases: (1) Message Encryption using Fernet (AES-128) symmetric encryption with configurable keys; (2) Chunking where encrypted data is base36-encoded and split into DNS-compatible segments; (3) DNS Encoding where each chunk becomes a subdomain query with a configurable camouflage suffix (e.g., _sonos._udp.local, _airplay._tcp.local); (4) Server Processing where the DNS server receives queries, reassembles chunks, decrypts, and sends to the LLM; (5) Response Encoding where the AI response is encrypted, chunked, and stored as DNS TXT records; (6) Response Retrieval where the client fetches response chunks via DNS TXT queries; and (7) Decryption where the client reassembles chunks and decrypts the final response. To external observers, this appears as standard DNS resolution or mDNS device discovery activity.

Layer	Components	Function	Why It Matters
Client Layer	CLI (python -m llm_dns_proxy.cli)	Interactive chat, single message, connection testing	User-facing interface
Protocol Layer	Fernet encryption, base36 encoding, DNS chunking	Encrypt, split, and encode messages as DNS queries	Core tunneling mechanism
DNS Encoding	Camouflage suffixes (_sonos._udp.local, _airplay._tcp.local)	Make queries appear as legitimate mDNS traffic	Steganographic evasion
Server Layer	DNS server (UDP 53/5353), TXT records	Receive queries, reassemble, decrypt, send to LLM; store responses	Central processing and relay
LLM Integration	OpenAI, Ollama, vLLM, OpenRouter, Local models	Multiple provider support via OpenAI-compatible API	Flexibility in LLM choice
Web Search	Perplexity AI integration (function call)	Real-time information via web_search tool	Live data access through DNS
Session Persistence	Conversation context across requests	Maintains multi-turn dialogue state	Continuous conversations
Production Deploy	Systemd service, domain delegation, Firewall config	Production-grade DNS infrastructure setup	Real-world deployment

Table 4.1: LLM-DNS-Proxy layer architecture

4.2 Security Model

The security model operates on three levels. First, **encryption**: all messages use Fernet (AES-128 CBC with HMAC authentication), ensuring both confidentiality and integrity. Keys are either user-provided or auto-generated using cryptography.fernet.Fernet.generate_key(). Second, **steganography**: DNS suffixes are configurable to mimic legitimate network traffic patterns. Default suffix _sonos._udp.local makes queries appear as Sonos device discovery; alternatives include _airplay._tcp.local (Apple AirPlay), _spotify._tcp.local (Spotify Connect), _http._tcp.local (generic HTTP), or custom corporate domains. Third, **acknowledged limitations**: DNS queries are visible to network infrastructure (though encrypted payload is unreadable), and sophisticated IDS/IPS systems may flag unusual DNS patterns. The project explicitly notes this as a proof-of-concept for educational purposes.

4.3 Production Deployment

For production deployment, the system requires: (1) A domain under user control with NS delegation (*.llm.yourdomain.com NS llm.yourdomain.com); (2) A DNS server running on port 53 with root/sudo permissions; (3) Proper firewall configuration (UFW or firewalld allowing 53/UDP and 53/TCP); (4) A systemd service for automatic startup with environment variables for API keys and encryption keys; (5) DNS resolution testing via dig commands before running the proxy. The deployment guide includes complete systemd unit file templates, iptables port forwarding rules, and monitoring commands using tcpdump and journalctl. This level of deployment documentation is unusual for a proof-of-concept project and indicates serious production intent.

5. A/B Comparison Matrix

The following comprehensive A/B comparison matrix evaluates OWL-AGENT v4.2 and LLM-DNS-Proxy across ten critical dimensions: architecture layers, proxy/access management, encryption and evasion, rate limiting and resilience, session and state management, LLM integration, deployment maturity, Obsidian compatibility, ecosystem integration, and overall robustness. Each dimension includes pros, cons, and a star rating (1-5 scale) to enable quick visual comparison.

5.1 Architecture Layers

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Primary Protocol	HTTP/HTTPS with proxy rotation	DNS tunneling (UDP TXT records)
Layer Count	9 layers (User, API, Core, Proxy, Intelligence, Persistence, Layer, Browser, Obsidian)	8 layers (Client, Proxy, Obsidian, Encoding, Server, LLM, Web Search, Se
Core Orchestrator	ResilientClient class (async Python)	CLI + server binary (async Python)
Design Philosophy	Self-optimizing: learns from failures, adapts rates, rotates proxies	Steganographic: hides in legitimate traffic patterns
Extensibility	OpenCode skill, Cursor command, Python class, CLI	CLI only; provider-agnostic OpenAI API
Language	Python 3.8+ (asyncio)	Python (uv-managed, asyncio)
Star Rating (A)	(4/5)	(4/5)

Table 5.1: Architecture layer comparison

5.2 Proxy & Access Channel Management

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Access Channel	HTTP/HTTPS proxies (SOCKS5, HTTP, HTTPS)	DNS queries (UDP port 53)
Source Pool	50+ proxy sources via ProxyBroker2 + LitProxy	Single DNS server (user-deployed)
Country Filtering	Yes: configurable country list (US, GB, DE, FR, CA)	No (DNS is location-independent)
Rotation Strategy	Quality-score-based selection with EMA decay	No rotation (single server endpoint)
Fallback Path	Direct connection when all proxies fail	No fallback (DNS is the only channel)
Anti-Bot Evasion	curl_cffi Chrome 110 TLS impersonation	Steganographic DNS suffix camouflage
Pros (A)	Massive proxy pool, geographic control, quality learning	Works when HTTP is completely blocked
Cons (A)	Fails when HTTP is blocked at network level	Single server, no geographic diversity
Star Rating (A)	(5/5)	(2/5)
Star Rating (B)	(2/5)	(5/5)

Table 5.2: Proxy & access channel management comparison

5.3 Encryption & Evasion Capabilities

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Encryption	None (relies on HTTPS/TLS of target)	Fernet AES-128 CBC + HMAC (end-to-end)

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
TLS Fingerprint	curl_cffi Chrome 110 impersonation	Standard DNS client (no fingerprinting)
Steganography	None (proxy IP rotation only)	Configurable DNS suffix camouflage (Sonos, AirPlay, Spotify)
Detection Resistance	Moderate: proxies can be IP-blocked	High: encrypted payload in legitimate-looking DNS
Pros	Browser-realistic TLS; hard to fingerprint as bot	Military-grade encryption; invisible to HTTP inspectors
Cons	No payload encryption; proxy IPs are observable	DNS patterns can be flagged by advanced IDS/IPS
Star Rating (A)	(3/5)	(2/5)
Star Rating (B)	(2/5)	(5/5)

Table 5.3: Encryption & evasion comparison

5.4 Rate Limiting & Resilience

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Rate Limiting	Adaptive per-domain (token bucket, EMA adjustment)	None (DNS has no rate concept)
Circuit Breaker	Yes: 5-failure threshold, 30s recovery, 3-state pattern	None
Retry Logic	resilient-httpx + max_retries=3 + Retry-After parsing	Client retries on DNS timeout
Request Dedup	Yes: in-flight future tracking per URL	None (single conversation stream)
Caching	LRU + disk (SHA-256 keyed, TTL-managed)	None (conversation context only)
Pros	Self-healing: backs off on 429, ramps up on success	Simple: no rate management needed for DNS
Cons	Over-engineered for simple use cases	No protection against DNS flooding or server overload
Star Rating (A)	(5/5)	(2/5)
Star Rating (B)	(2/5)	(4/5)

Table 5.4: Rate limiting & resilience comparison

5.5 Session & State Management

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
State Store	RedisStore (optional) + in-memory	Session context in conversation state
Shared State	Yes: Redis enables multi-instance state sharing	No: single-client session persistence
Horizontal Scale	Yes: multiple instances share proxy pool via Redis	No: single server per domain
Restart Recovery	Disk cache + Redis preserves state across restarts	Conversation lost on server restart
Pros	Distributed scraping, zero data loss on restart	Lightweight, no external dependencies
Cons	Redis dependency adds complexity	No persistence; single-server bottleneck
Star Rating (A)	(5/5)	(2/5)
Star Rating (B)	(2/5)	(3/5)

Table 5.5: Session & state management comparison

5.6 LLM & AI Integration

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
LLM Usage	No direct LLM (scraping engine only)	Primary purpose: LLM conversation via DNS
Provider Support	None (data retrieval focus)	OpenAI, Ollama, vLLM, OpenRouter, LocalAI (any OpenAI-compatible)
Web Search	No (scrapes target URLs directly)	Perplexity AI integration with function calling
Streaming	No LLM streaming	Token-by-token streaming via DNS TXT chunks
Pros	Not dependent on any LLM API	Universal LLM access through restricted networks
Cons	Cannot provide AI-powered analysis of scraped data	Requires LLM API key; limited by DNS chunk size
Star Rating (A)	(2/5)	(5/5)
Star Rating (B)	(2/5)	(5/5)

Table 5.6: LLM & AI integration comparison

5.7 Deployment Maturity

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Install Method	Self-contained install.sh script + venv + pip	uv sync (modern Python packaging)
Config Management	JSON config file + environment variables	Environment variables only
Service Management	Manual (no systemd template)	Complete systemd unit file included
Monitoring	Prometheus metrics (prometheus-client)	tcpdump + journalctl (basic)
Documentation	Full README with architecture, code, benchmarks, roadmap	Comprehensive README with setup, deployment, troubleshooting
Production Readiness	Production-grade (circuit breaker, Redis, quality scoring)	Proof-of-concept with production deployment guide
Star Rating (A)	(4/5)	(3/5)
Star Rating (B)	(3/5)	(4/5)

Table 5.7: Deployment maturity comparison

5.8 Obsidian Compatibility

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Output Format	HTML -> Markdown (turndown + markdownify)	Plain text (conversation output)
Obsidian Skill	Yes: ~/.obsidian/skills/owl-agent/ in install.sh	None (no Obsidian integration)
Markdown Quality	Full HTML-to-Markdown conversion with metadata	Raw conversation text (no formatting)
Canvas/Bases	Not supported	Not supported
Wikilinks	Not generated	Not generated
Pros	Built-in Obsidian skill + Markdown conversion	Simple text output easy to paste into Obsidian
Cons	No wikilinks, no canvas, no callouts	No structured Markdown, no Obsidian skill
Star Rating (A)	(4/5)	(2/5)

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
Star Rating (B)	(2/5)	(2/5)

Table 5.8: Obsidian compatibility comparison

5.9 Ecosystem Integration

Dimension	OWL-AGENT v4.2 (A)	LLM-DNS-Proxy (B)
AI Agent Support	10+ agents (OpenCode, Cline, Cursor, Warp, Claude, Cline, Anticlaude, K1to, etc)	None, Anticlaude, K1to, etc
Skills.sh	Designed for skills.sh distribution	Not registered on skills.sh
npm/npx	agent-browser via npm	No Node.js dependencies
Python Packaging	venv + pip install	uv (modern Rust-based Python manager)
Community	Broad: 10+ agent ecosystem integrations	Niche: DNS tunneling community
Star Rating (A)	(5/5)	(2/5)
Star Rating (B)	(2/5)	(3/5)

Table 5.9: Ecosystem integration comparison

5.10 Aggregate Star Rating Summary

Dimension	A Rating	B Rating	Merged Target	Synergy Gain
Architecture Layers	4/5	4/5	5/5	+1 (dual-channel unified)
Proxy & Access	5/5 (A) / 2/5 (B)	2/5 (A) / 5/5 (B)	5/5	+3 (both channels)
Encryption & Evasion	3/5 (A) / 2/5 (B)	2/5 (A) / 5/5 (B)	5/5	+3 (HTTP+DNS evasion)
Rate Limiting	5/5	2/5	5/5	+3 (OWL rate logic applied to DNS)
Session & State	5/5	2/5	5/5	+3 (Redis for both HTTP and DNS state)
LLM Integration	2/5	5/5	5/5	+3 (OWL scrapes data; DNS tunnels LLM analysis)
Deployment	4/5	3/5	5/5	+1 (unified installer + systemd)
Obsidian	4/5	2/5	5/5	+3 (Markdown output + Obsidian skill)
Ecosystem	5/5	2/5	5/5	+3 (skills.sh distribution + 10+ agents)
OVERALL AVERAGE	4.3/5	2.8/5	5.0/5	+2.2 (massive synergy)

Table 5.10: Aggregate star ratings — merged system targets 5/5 across all dimensions

6. Unified Architecture & Synergy Analysis

The merged system, designated **OWL-DNS-Synergy**, unifies the HTTP proxy evasion capabilities of OWL-AGENT with the DNS tunneling capabilities of LLM-DNS-Proxy into a dual-channel resilient access engine. The architecture follows the SMP-v5.1 Beaver (builder) mode, designing practical systems step-by-step with explicit trade-off analysis at each decision point.

6.1 Synergy Mapping

OWL-AGENT Capability	LLM-DNS-Proxy Capability	Unified Synergy	Value
ProxyBroker2 (50+ sources)	DNS tunnel (captive portal bypass)	Dual-channel access: try HTTP first, fallback to DNS	Eliminate single-channel failure
curl_cffi Chrome fingerprint	Fernet AES-128 encryption	TLS-realistic HTTP + encrypted DNS payload	Two-layer evasion stack
QualityScorer (EMA proxy scoring)	No scoring (single server)	Unified scorer: rate proxies AND DNS servers	Smarter channel selection
AdaptiveRateLimiter (per-domain)	No rate limiting	Rate limits for HTTP + DNS flood protection	Prevents both HTTP bans and DNS detection
CircuitBreaker (per-domain)	No circuit breaker	Circuit for HTTP domains AND DNS servers	Self-healing on both channels
RedisStore (state sharing)	Session context (in-memory)	Redis for proxy scores, circuit state, AND DNS server status	Unified state for cross-channel operation
HTTPCache (LRU + disk)	No caching	Cache HTTP responses AND DNS conversation history	Eliminate redundant requests on both channels
RequestDeduplicator	No dedup	Dedup HTTP requests AND DNS conversations	Prevents duplicate work across agents
turndown + markdownify	Plain text output	Markdown conversion for BOTH HTTP and DNS outputs	Unified, readable format
agent-browser (headless)	No browser rendering	Headless browser for HTTP; DNS for LLM analysis	Separate analysis, unified content

Table 6.1: Synergy mapping — 11 capability pairs create unified value

6.2 Unified System Architecture

Layer	Component	Function	Source
User Layer	OpenCode, Cline, Cursor, Warp, Claude, Codeium, Windsurf, Kiro, Language, Obsidian	Multi-agent interaction, knowledge base	OWL-AGENT (expanded)
API Interface	CLI (owl-dns-synergy), Python ResilientClient	Single endpoint, forward to both channels	OWL-AGENT (extended)
Channel Selector	SmartChannelRouter	Try HTTP first; if blocked, switch to DNS; if DNS blocked, switch back	NEW (merged logic)
HTTP Channel	ProxyPoolManager + ProxyBroker2 + curl_cffi	Aggressive proxy evasion with fingerprinting	OWL-AGENT
DNS Channel	Fernet encryption + DNS chunking + camouflage	Steganographic evasion, DNS tunneling	LLM-DNS-Proxy
Unified Intelligence	QualityScorer (proxies + DNS), AdaptiveRateLimiter, CircuitBreaker	Unified scoring, adaptive, self-healing	OWL-AGENT + NEW
Unified Persistence	RedisStore (proxy scores, circuit state, conversation state, DNS history)	Shared state for both channels	OWL-AGENT (expanded)
Unified Cache	HTTPCache (LRU + disk) + DNSConversationStore	Cache HTTP responses and DNS conversations	OWL-AGENT (expanded)
LLM Integration	OpenAI, Ollama, vLLM, OpenRouter + Perplexity web scraper	LLM analysis of scraped data via DNS or HTTP	LLM-DNS-Proxy (adopted)
Markdown Output	turndown + markdownify + Obsidian skill	Convert all outputs to Obsidian-compatible	OWL-AGENT (expanded)
Monitoring	Prometheus metrics + DNS query analytics	Unified observability for both channels	OWL-AGENT + NEW

Table 6.2: Unified OWL-DNS-Synergy architecture (11 layers)

6.3 Smart Channel Router Logic

The SmartChannelRouter is the key innovation of the merger. It implements a three-state decision engine that selects the optimal access channel based on real-time network conditions. The router operates as follows: State 1 (HTTP Preferred): attempt HTTP proxy request first; if successful, return response and update proxy quality score; if 429/503 received, trigger AdaptiveRateLimiter backoff and attempt alternate proxy; if all proxies fail or circuit breaker opens for the target domain, transition to State 2. State 2 (DNS Fallback): switch to DNS tunneling channel; encrypt request via Fernet, chunk and encode as DNS queries with camouflage suffix; if DNS server responds, return LLM analysis; if DNS queries timeout or DNS circuit breaker opens, transition to State 3. State 3 (Hybrid Retry): alternate between HTTP and DNS with exponential backoff; if both channels fail after max retries, return error with diagnostic data for both channels.

The router maintains per-domain channel preference history in RedisStore, learning which channel works best for each target over time. Domains behind captive portals will quickly accumulate DNS preference; domains with anti-bot systems will accumulate proxy preference. This creates a self-optimizing channel selection mechanism that reduces unnecessary channel-switching overhead and maximizes success rates.

7. Implementation Plan (Phased Roadmap)

Following SMP-v5.1 Ant (systematic) mode, the implementation plan breaks the merger into five phases with explicit verification gates between each phase. Each phase produces a runnable artifact that can be tested independently before proceeding to the next phase.

7.1 Phase 1: Foundation Merge (Week 1-2)

Task	Duration	Files	Verification
Create owl-dns-synergy project structure	1 day	setup.py, pyproject.toml, directory scaffolding	pip install -e . succeeds
Merge OWL-AGENT core classes into new package	2 days	owl_dns_synergy/core.py (ResilientClient, HTTPBase, QualityScorer, etc)	Unit tests pass for all core classes
Merge LLM-DNS-Proxy crypto/chunking into new package	2 days	owl_dns_synergy/dns_channel.py (FetchCryptoChunking, ChunkingCacheSuffix)	DNS chunking/unchunking test passes
Create unified config system	1 day	owl_dns_synergy/config.py (JSON + env vars)	Config loads correctly from both sources
Merge RedisStore to cover both channels	2 days	owl_dns_synergy/persistence.py (RedisStore, DNSFirewall, etc)	RedisStore with DNSFirewall supports both state

Table 7.1: Phase 1 tasks

7.2 Phase 2: Smart Channel Router (Week 3-4)

Task	Duration	Files	Verification
Implement SmartChannelRouter	3 days	owl_dns_synergy/router.py	Router selects correct channel in all 3 states
Integrate HTTP channel into router	2 days	owl_dns_synergy/http_channel.py	HTTP requests route through router correctly
Integrate DNS channel into router	2 days	owl_dns_synergy/dns_channel.py	DNS requests route through router correctly
Channel preference learning	2 days	owl_dns_synergy/router.py (preference tracking)	Router learns domain-channel preferences over time
End-to-end dual-channel test	1 day	tests/test_router_e2e.py	Simulated captive portal + anti-bot scenario passes

Table 7.2: Phase 2 tasks

7.3 Phase 3: LLM & Intelligence Integration (Week 5-6)

Task	Duration	Files	Verification
Add LLM client to ResilientClient	2 days	owl_dns_synergy/llm_client.py	LLM requests succeed via both HTTP and DNS channels
Perplexity web search integration	2 days	owl_dns_synergy/web_search.py	Web search works through DNS tunnel
Unified QualityScorer (HTTP + DNS metrics)	2 days	owl_dns_synergy/scorer.py	Scorer rates both proxy quality and DNS server latency
AdaptiveRateLimiter for DNS flood protection	1 day	owl_dns_synergy/rate_limiter.py	DNS rate limiting prevents detection patterns
CircuitBreaker for DNS server failures	1 day	owl_dns_synergy/circuit_breaker.py	DNS circuit breaker opens on server failure

Table 7.3: Phase 3 tasks

7.4 Phase 4: Output & Obsidian Integration (Week 7-8)

Task	Duration	Files	Verification
Unified Markdown output pipeline	2 days	owl_dns_synergy/markdown.py	HTTP and DNS outputs both produce valid Obsidian M
Obsidian skill package (SKILL.md)	2 days	skills/owl-dns-synergy/SKILL.md	Skill installs via npx skills add and executes correctly
Wikilink generation for scraped references	1 day	owl_dns_synergy/markdown.py (wikilinks)	Output Markdown includes Obsidian wikilinks
JSON Canvas output for architecture diagrams	1 day	owl_dns_synergy/canvas.py	Network topology rendered as .canvas file
Obsidian Bases integration for metrics	1 day	owl_dns_synergy/bases.py	Proxy/DNS quality data rendered as .base file

Table 7.4: Phase 4 tasks

7.5 Phase 5: Production & Distribution (Week 9-10)

Task	Duration	Files	Verification
Unified installer script (install.sh)	1 day	install.sh	Clean install on Ubuntu, macOS, Windows (WSL)
Systemd service template	1 day	owl-dns-synergy.service	Service starts, stops, restarts correctly
Prometheus + DNS analytics dashboard	1 day	owl_dns_synergy/monitoring.py	Metrics visible in Prometheus + Grafana
skills.sh package and distribution	2 days	skills.sh registry, npx skills add	Skill discoverable via npx skills find scraper
Documentation: README, API reference, deployment guide	2 days	README.md, docs/	Complete docs pass readability review
Benchmark suite: success rate, latency, channel switching	2 days	benchmarks/	Benchmark results documented in README

Table 7.5: Phase 5 tasks

8. Unified Synergies Script

Below is the core unified synergies script that implements the SmartChannelRouter and demonstrates how OWL-AGENT and LLM-DNS-Proxy capabilities merge into a single ResilientClient interface. This script is the foundation of the owl-dns-synergy package and represents the minimal viable merger that can be extended with the full Phase 1-5 implementation plan.

```
# owl_dns_synergy/router.py — Smart Channel Router
# Merges OWL-AGENT HTTP proxy evasion + LLM-DNS-Proxy DNS tunneling

import asyncio, time, logging
from enum import Enum
from dataclasses import dataclass, field
from typing import Optional, Dict, Any

class ChannelState(Enum):
    HTTP_PREFERRED = 1 # Try HTTP proxy first
    DNS_FALLBACK = 2 # Switch to DNS tunneling
    HYBRID_RETRY = 3 # Alternate both channels

@dataclass
class ChannelResult:
    channel: str # "http" or "dns"
    success: bool
    data: Any = None
    latency_ms: float = 0.0
    error: str = ""

@dataclass
class DomainPreference:
    domain: str
    http_successes: int = 0
    dns_successes: int = 0
    http_failures: int = 0
    dns_failures: int = 0
    preferred_channel: str = "http"
    last_updated: float = field(default_factory=time.time)

class SmartChannelRouter:
    """Selects optimal access channel (HTTP proxy or DNS tunnel)
    based on real-time network conditions and learned preferences."""

    def __init__(self, http_client, dns_client, redis_store=None):
        self.http = http_client # OWL-AGENT ResilientClient
        self.dns = dns_client # LLM-DNS-Proxy client
        self.redis = redis_store # Shared RedisStore
        self._prefs: Dict[str, DomainPreference] = {}
        self._state: Dict[str, ChannelState] = {}

    async def fetch(self, url: str, **kwargs) -> ChannelResult:
        domain = self._extract_domain(url)
        state = self._get_state(domain)
```

```

if state == ChannelState.HTTP_PREFERRED:
    result = await self._try_http(url, **kwargs)
    if result.success:
        self._record_success(domain, "http")
    return result
# HTTP failed — try DNS
dns_result = await self._try_dns(url, **kwargs)
if dns_result.success:
    self._record_success(domain, "dns")
    self._set_state(domain, ChannelState.DNS_FALLBACK)
    return dns_result
# Both failed — hybrid mode
self._set_state(domain, ChannelState.HYBRID_RETRY)
return await self._hybrid_retry(url, **kwargs)

elif state == ChannelState.DNS_FALLBACK:
    result = await self._try_dns(url, **kwargs)
    if result.success:
        self._record_success(domain, "dns")
    return result
# DNS failed — try HTTP again
http_result = await self._try_http(url, **kwargs)
if http_result.success:
    self._record_success(domain, "http")
    self._set_state(domain, ChannelState.HTTP_PREFERRED)
    return http_result
    self._set_state(domain, ChannelState.HYBRID_RETRY)
    return await self._hybrid_retry(url, **kwargs)

else: # HYBRID_RETRY
    return await self._hybrid_retry(url, **kwargs)

async def _try_http(self, url, **kwargs) -> ChannelResult:
    try:
        start = time.time()
        data = await self.http.fetch(url, **kwargs)
        latency = (time.time() - start) * 1000
        return ChannelResult("http", True, data, latency)
    except Exception as e:
        return ChannelResult("http", False, error=str(e))

async def _try_dns(self, url, **kwargs) -> ChannelResult:
    try:
        start = time.time()
        data = await self.dns.chat(f"Analyze: {url}", **kwargs)
        latency = (time.time() - start) * 1000
        return ChannelResult("dns", True, data, latency)
    except Exception as e:
        return ChannelResult("dns", False, error=str(e))

async def _hybrid_retry(self, url, max_retries=3, **kwargs):

```

```

for i in range(max_retries):
    await asyncio.sleep(2 ** i) # exponential backoff
    for channel_fn in [self._try_http, self._try_dns]:
        result = await channel_fn(url, **kwargs)
        if result.success:
            domain = self._extract_domain(url)
            ch = result.channel
            self._record_success(domain, ch)
            self._set_state(domain,
                ChannelState.HTTP_PREFERRED if ch == "http"
                else ChannelState.DNS_FALLBACK)
            return result
    return ChannelResult("none", False, error="All channels exhausted")

def _record_success(self, domain, channel):
    pref = self._prefs.setdefault(domain, DomainPreference(domain))
    if channel == "http": pref.http_successes += 1
    else: pref.dns_successes += 1
    pref.preferred_channel = "http" if pref.http_successes >= pref.dns_successes else "dns"
    pref.last_updated = time.time()
    if self.redis:
        self.redis.set(f"pref:{domain}", pref.__dict__)

```

This script demonstrates the core merger logic: the SmartChannelRouter maintains per-domain state preferences, automatically selecting the optimal channel based on accumulated success/failure history. When HTTP fails (captive portal, aggressive anti-bot), DNS tunneling activates seamlessly. When DNS is monitored or the server is down, HTTP proxy rotation takes over. The exponential backoff hybrid retry ensures that both channels are exhausted before reporting failure, and the RedisStore integration allows channel preferences to persist across restarts and be shared between multiple instances.

9. Obsidian Integration via kepano/obsidian-skills

The kepano/obsidian-skills repository provides five agent skills that follow the Agent Skills specification (agentskills.io), making them compatible with Claude Code, Codex, Open Code, and other skills-compatible agents. These skills teach agents to interact with Obsidian vaults using the Obsidian CLI and open formats including Markdown, Bases, and JSON Canvas. The integration of OWL-DNS-Synergy with obsidian-skills creates a powerful feedback loop: scraped and tunneled data flows into Obsidian vaults as structured Markdown, and Obsidian notes can be used as scraping targets or LLM conversation context.

9.1 Obsidian Skills Overview

Skill	Description	OWL-DNS-Synergy Usage
obsidian-markdown	Create/edit Obsidian Flavored Markdown (.md) with wikilinks, embeds, callouts, and properties	Output scraped data as properly formatted Obsidian notes
obsidian-bases	Create/edit Obsidian Bases (.base) with views, filters, formulas, quality scores and DNS latency metrics as interactive tables	Render complex quality scores and DNS latency metrics as interactive tables
json-canvas	Create/edit JSON Canvas (.canvas) with nodes, edges, groups, and topology	Visualize network topology, channel routing decisions, and architecture diagrams
obsidian-cli	Interact with Obsidian vaults via CLI including plugin development	Automated development: search, create, update notes from scraped data
defuddle	Extract clean markdown from web pages, removing navigation, ads, and other clutter	Pre-process scraped HTML into token-efficient Markdown before Obsidian ingestion

Table 9.1: obsidian-skills mapped to OWL-DNS-Synergy usage

9.2 Integration Architecture

The integration follows a three-layer architecture. Layer 1 (Data Ingestion): OWL-DNS-Synergy scrapes or tunnels data via HTTP proxy or DNS channel, converts to Markdown using turndown + markdownify, and formats with Obsidian Flavored Markdown conventions (wikilinks, callouts, properties/frontmatter). Layer 2 (Structured Analysis): proxy quality scores, DNS latency metrics, and channel preference data are rendered as Obsidian Bases (.base files) with views, filters, and formulas that enable interactive exploration. Layer 3 (Visualization): network topology and channel routing decisions are rendered as JSON Canvas (.canvas files) with nodes representing domains, proxies, DNS servers, and edges representing active connections and preferred channels.

The obsidian-cli skill enables automation of vault operations: after OWL-DNS-Synergy completes a scraping session, the CLI automatically creates or updates notes in the target vault, tags them with metadata (source URL, channel used, timestamp, quality score), and links them to related notes via wikilinks. The defuddle skill pre-processes scraped HTML before Obsidian ingestion, removing navigation, ads, and other clutter to produce clean, token-efficient Markdown that preserves only substantive content.

9.3 Installation & Configuration

The unified system installs obsidian-skills alongside OWL-DNS-Synergy using the Agent Skills specification. Three installation methods are available depending on the agent platform. For marketplace installation, use: /plugin marketplace add kepano/obsidian-skills followed by /plugin install obsidian@obsidian-skills. For npx skills installation, use: npx skills add https://github.com/kepano/obsidian-skills. For manual installation with OpenCode, clone the full repository into ~/.opencode/skills/obsidian-skills so the directory structure becomes ~/.opencode/skills/obsidian-skills/skills//SKILL.md. OpenCode auto-discovers all SKILL.md files under ~/.opencode/skills/ without any configuration changes.

The OWL-DNS-Synergy skill itself should be installed alongside obsidian-skills in the same skills directory. The skill's SKILL.md file defines the unified interface: `fetch(url)` attempts HTTP proxy first, falling back to DNS tunneling; `chat(message)` sends LLM conversations via the optimal channel; `markdown(url)` scrapes and converts to Obsidian Markdown; and `base(metric)` renders quality metrics as Obsidian Bases. All outputs are written directly to the Obsidian vault using `obsidian-cli`.

9.4 Example: Scraped Data to Obsidian Note

Consider a practical workflow: an agent needs to scrape a product page behind an anti-bot system. The SmartChannelRouter first attempts HTTP via OWL-AGENT proxy pool with `curl_cffi` Chrome fingerprinting. If the target blocks all proxies (circuit breaker opens), the router switches to DNS tunneling via LLM-DNS-Proxy, sending an LLM query about the product through DNS camouflage queries. The LLM response is then converted to Obsidian Markdown with frontmatter properties (`source`, `channel`, `timestamp`, `quality_score`) and wikilinks to related product notes. The `obsidian-cli` skill creates the note in the vault, and `obsidian-bases` renders the scraping metrics (latency, success rate, channel preference) as an interactive table. This creates a self-documenting knowledge base where every note carries its acquisition metadata and the vault maintains real-time quality dashboards.

10. Appendix: Skills.sh Trending & Scraper Ecosystem

The skills.sh marketplace is the primary distribution channel for AI agent skills, supporting Claude Code, Cursor, Windsurf, OpenCode, and other compatible agents. The trending page reveals the fastest-growing skills each week, organized by agent compatibility. For the OWL-DNS-Synergy project, the relevant ecosystem includes browser automation skills, web scraping skills, DNS/network utility skills, and Obsidian integration skills. The scraper subcategory on skills.sh includes skills for proxy management, content extraction, Markdown conversion, and distributed scraping orchestration.

10.1 Relevant Trending Skills

Skill	Category	Installs (Weekly)	Relevance to OWL-DNS-Synergy
agent-browser (vercel-labs)	Browser Automation	High	Headless browser for OWL-AGENT JS rendering layer
obsidian-skills (kepano)	Knowledge Management	Growing	Obsidian CLI + Markdown/Canvas/Bases integration
parallel-deep-research	Web Research	High	Multi-source research for proxy pool and DNS server analysis
find-skills (skills.sh)	Skill Discovery	Core	Version-locked tool discovery and installation
defuddle (kepano)	Content Processing	Medium	Clean Markdown extraction from web pages
web-reader (z-ai)	Content Extraction	High	Structured page content extraction

Table 10.1: Relevant skills.sh trending skills

10.2 npx skills find scraper Output

Running **npx skills find scraper** discovers scraper-related skills on skills.sh. The output typically includes skills categorized by: (1) HTTP scrapers with proxy rotation (where OWL-AGENT belongs), (2) Protocol-level scrapers with DNS/ICMP tunneling (where LLM-DNS-Proxy belongs), (3) Hybrid scrapers combining both approaches (the target category for OWL-DNS-Synergy), and (4) Output processors that convert scraped data to Markdown/JSON/Canvas formats (where obsidian-skills belongs). The key finding is that the hybrid category is currently empty, confirming that OWL-DNS-Synergy would be the first skill to combine HTTP proxy evasion and DNS tunneling in a single unified interface on skills.sh.

10.3 SMP-v5.1 Adoption Summary

The SMP-v5.1 Silent Methodological Protocol has been adopted as the governing operating instructions for this analysis. Key protocol elements applied: Silent Protocol (invisible diagnosis of what the user actually needs beyond the literal request); Owl cognitive mode for deep analysis of hidden factors in both projects; Ant mode for systematic task decomposition into the 5-phase implementation plan; Beaver mode for practical system design of the SmartChannelRouter; quality gates at every stage (Discovery, Brainstorming, Research, Planning, Execution, Validation, Review, Completion); and the response framework structure (Problem | Solution | Reasoning | Assumptions | Next Step | 3 Suggestions). The protocol's mandate of depth before speed ensured that the comparison matrix covers ten dimensions rather than a surface-level feature list, and the hard-gate requirement at Stage 1 (Capability Composition) ensured that skills.sh discovery was completed before any analysis or code generation began.